00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D

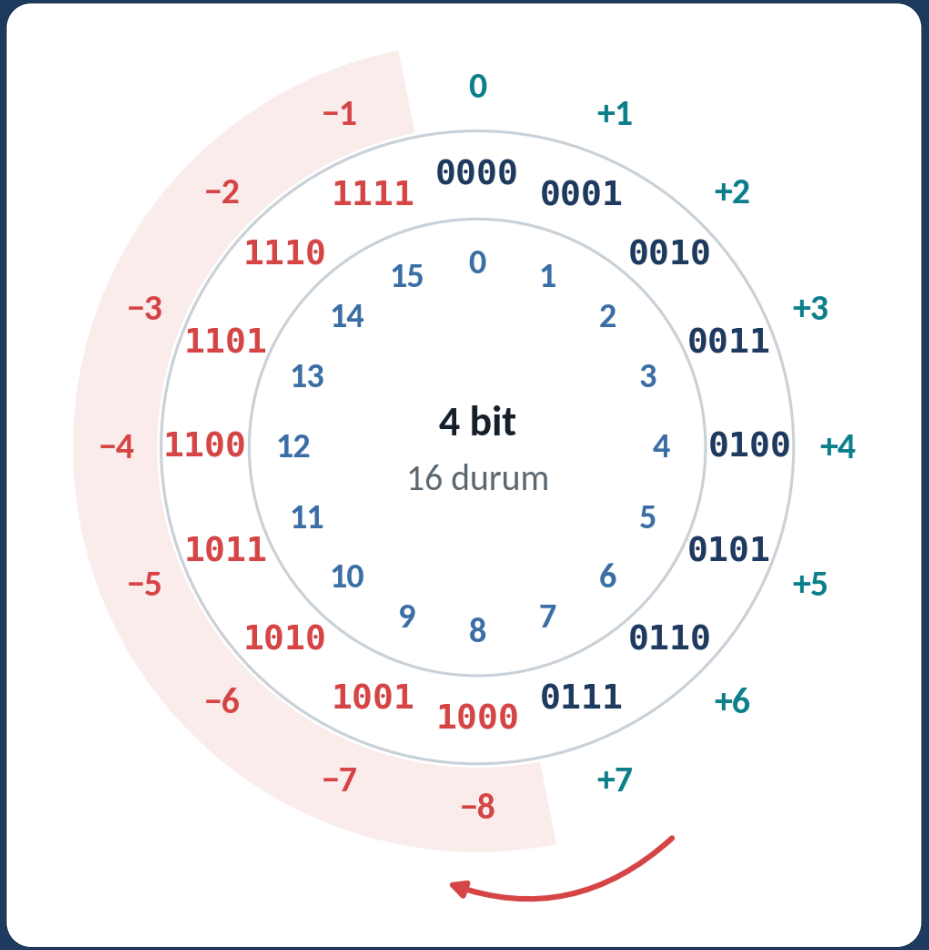
ALGORİTMA VE PROGRAMLAMA – I

HAFTA 03

C’nin Yapı Taşları: Değişkenler, Veri Tipleri, Bellekte Temsil ve Temel Giriş/Çıkış

Bir değişken bellekte nasıl yaşar, sayılar nasıl bitlere dönüşür, klavyeden nasıl okuyup ekrana yazarız?

MODÜL II · VERİNİN TEMSİLİ VE İŞLENMESİ



Dr. Öğr. Üyesi Mehmet Sevri · RTEÜ Bilgisayar Mühendisliği · 2026–2027 Güz

14 HAFTALIK YOLCULUK

Yol haritasında neredeyiz?

H1	H2	H3	H4	H5	H6	H7
Bilgisayar ve Program	Algoritmik Düşünme	**Değişkenler ve Veri Tipleri**	Operatörler ve İfadeler	Karar Yapıları	Döngüler I	Döngüler II + Ara Proje
0x00	0x01	0x02	0x03	0x04	0x05	0x06

H8	H9	H10	H11	H12	H13	H14
Fonksiyonlar I	Kapsam, Yığın, Özyineleme	Diziler I	Diziler II: Matrisler	İşaretçiler ve Bellek	Karakter Dizileri	Yapılar, Dosyalar, Final
0x07	0x08	0x09	0x0A	0x0B	0x0C	0x0D

Vize: H1–H7 · Ara proje: H7 · Final projesi: H14

GEÇEN HAFTADAN

Geçen haftadan · S1: “Çaya biraz şeker ekle.” adımı hangi özelliği ihlal eder?

A Sonluluk

B Belirlilik

C Girdi

D Etkinlik

GEÇEN HAFTADAN

Geçen haftadan · S2: Döngü bittiğinde toplam ve i kaçtır?

sözde kod (H2)

```
1  toplam ← 0
2  i ← 1
3  i ≤ 3 olduğu sürece:
4      toplam ← toplam + i
5      i ← i + 1
6  YAZ toplam, i
```

A toplam = 6, i = 3

B toplam = 6, i = 4

C toplam = 3, i = 3

D toplam = 10, i = 4

Geçen haftadan · S3 ve yanıtlar

S3: Akış diyagramında “kullanıcıdan sayı oku” adımı hangi sembolle çizilir? Önce yanıtlayın, sonra kartlara bakın.

S1 · Belirlilik (B)

“Biraz” ölçülemez; iki kişi iki farklı miktar koyar. Düzeltme: “1 çay kaşığı şeker ekle.”

S2 · toplam = 6, i = 4 (B)

Son denetimde `4 ≤ 3` yanlış olur ve döngü biter. `toplam ← 0` unutulsaydı? → bu haftanın **Hata Avcısı 3**’ü.

S3 · Paralelkenar

Giriş/çıkış sembolü. Bu hafta: okuma → `scanf`, yazma → `printf`. H2’deki “etiketli kutu” artık bir **değişken**, ← ise **=** (atama).

BÜYÜK SORU

Haftanın Büyük Sorusu

Bilgisayar neden 0.1 + 0.2 işleminin sonucunu tam olarak 0.3 bulamaz ve bir sayaç neden birdenbire eksi değere düşer?

Bilgisayar: 0.1 + 0.2 = ?

GERÇEK ÇIKTI · H03_12

```
toplam (%f)      : 0.300000
toplam (%.20f)   : 0.30000000000000004441
0.3 (%.20f)      : 0.29999999999999998890
0.3'e eşit mi?   : 0 (1 evet, 0 hayır)
```

Bilgisayar: 2 147 483 647 + 1 = ?

GERÇEK ÇIKTI · H03_11

```
int en büyük          : 2147483647
INT_MAX + 1           : -2147483648
```

*İşaretli taşma C'de **tanımsız davranıştır**; bu, bu makinedeki gözlemdir.*

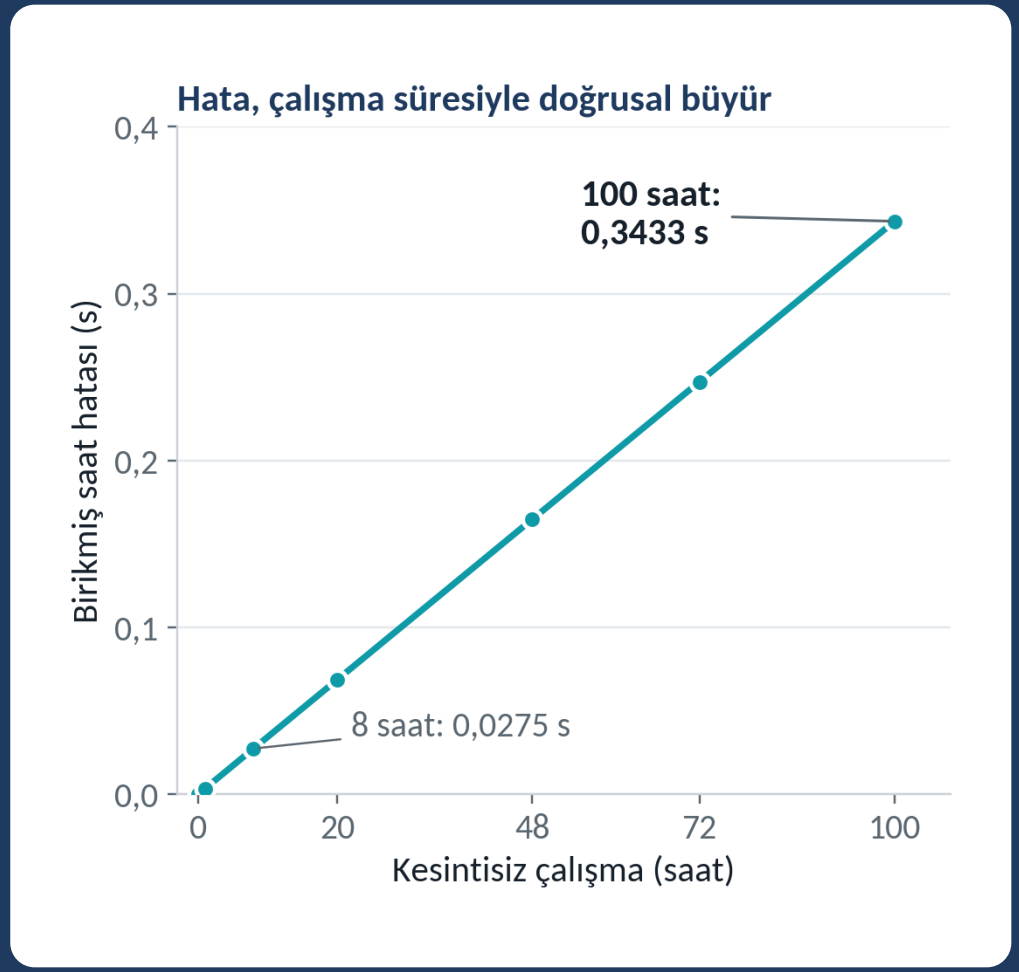
Bilgisayar yanlış mı hesaplıyor? Hayır: **sonlu bitle** çalışıyor. Bu hafta her iki sonucu da bit düzeyinde açıklayacağız.

GERÇEK DÜNYADAN

25 Şubat 1991 · Dhahran

0,1 saniyenin bedeli: Dhahran, 1991

- 25 Şubat 1991'de bir Patriot bataryası yaklaşan Scud füzesini önleyemedi; kışlaya isabet eden füze **28 Amerikan askerinin** ölümüne yol açtı.
- Batarya 100 saati aşkın süredir kesintisiz çalışıyordu. Saat, zamanı 0,1 s'lik tıklarla sayıyor ve 0,1 ile çarpıyordu.
- 0,1 ikili tabanda sonlu yazılamaz; 24 bitlik yazmaçta kesildi. Her tıkta ≈ $9{,}5 \times 10^{-8}$ s hata → 100 saatte **≈ 0,34 s** sapma.



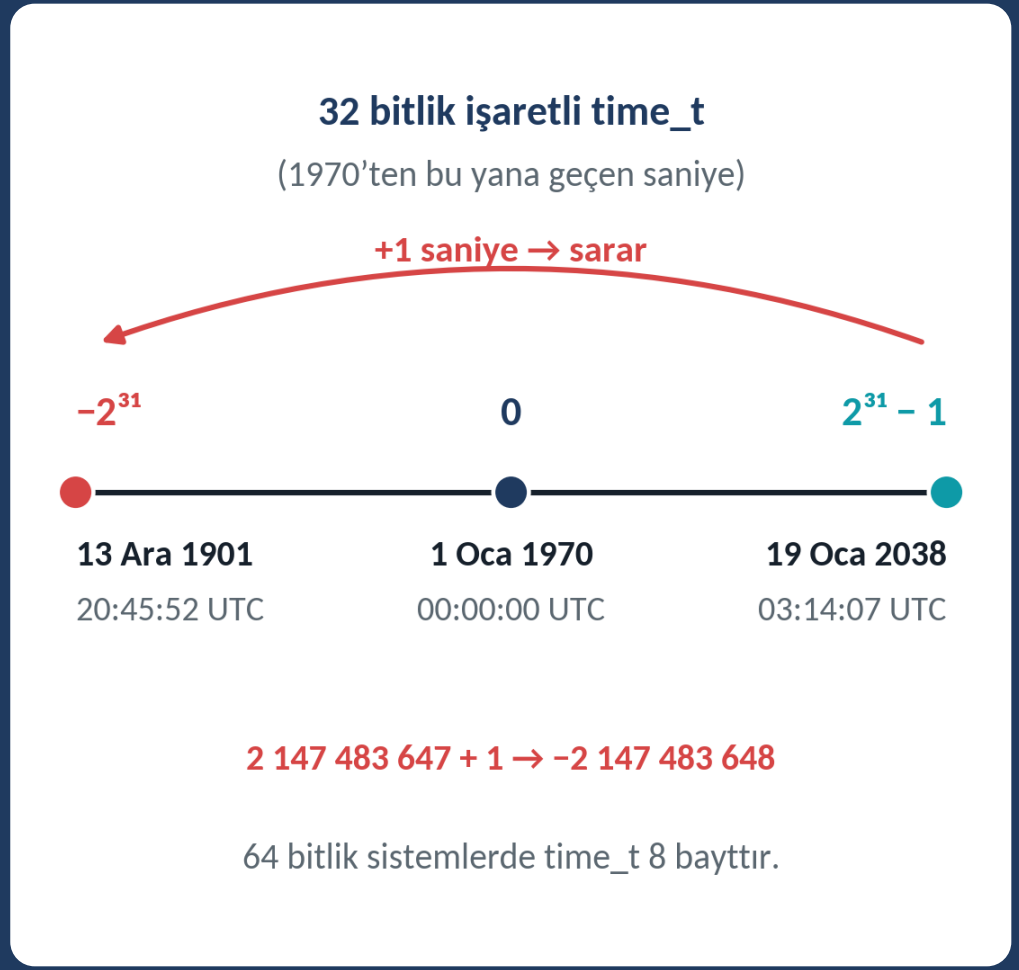
Kaynak: GAO (1992), GAO/IMTEC-92-26 · Grafik: kendi programımızın (H03_16) çıktısı

GERÇEK DÜNYADAN

2014 · 2038

Sınırsız sayaç yoktur: Gangnam Style ve 2038

- Aralık 2014: “Gangnam Style” görüntülenme sayısı **2 147 483 647**’yi ($2^{31} - 1$) aşan ilk YouTube videosu oldu; YouTube sayacı 64 bite yükseltti.
- 32 bitlik işaretli `time_t` sayacı 19 Ocak 2038 03:14:07 UTC’de dolar; bir sonraki saniyede tarih pratikte 1901’e “düşer”.
- Ortak nokta: sabit sayıda bit = **sınırlı aralık**. Doğru tip seçimi bir mühendislik kararıdır.



Kaynak: Guinness World Records (t.y.) · 64 bitlik Linux’ta time_t 8 bayttır

ÖĞRENME ÇIKTILARI

Bu hafta sonunda neler yapabileceksiniz?

ÖÇ 3.1	Bir C programının yapısını (önişlemci yönergeleri, main fonksiyonu, deyimler, yorumlar) tanımlar.
ÖÇ 3.2	Değişkeni ad, tip, değer ve adres nitelikleriyle açıklar; bildirim, ilk değer atama ve atama işlemlerini doğru kullanır.
ÖÇ 3.3	Temel veri tiplerinin boyutlarını ve değer aralıklarını belirler; probleme uygun tipi gerekçelendirerek seçer.
ÖÇ 3.4	İkiye tümleyen gösterimi ve IEEE 754 kayan nokta gösterimini sezgisel düzeyde açıklar; taşma ve hassasiyet kaybını öngörür.
ÖÇ 3.5	printf ve scanf ile biçimlendirilmiş giriş/çıkış yapar.

01

Değişken: adı, tipi, değeri ve adresi

Bir C programının parçaları, tanımlayıcılar ve bellek kutusu gösterimi

Bir C programının anatomisi

```c
/* Yaş programı */
#include <stdio.h>

int main(void)
{
    int yas = 19;
    yas = yas + 1;
    printf("Yaş: %d\n", yas);
    return 0;
}
```

- Yorum: derleyici yok sayar
- Önişlemci yönergesi: stdio.h eklenir
- main: programın giriş noktası
- { … }: main'in gövdesi (blok)
- Bildirim + ilk değer atama
- Atama deyimi
- Fonksiyon çağrısı (çıktı)
- return 0: işletim sistemine "başarılı"

- **Önişlemci**: `#include` derlemeden önce işlenir; sonunda `;` yok.
- **main**: programın giriş noktası; `return 0;` → "başarılı".
- **Tanımlayıcı** kuralları: harf, rakam, _; rakamla başlamaz; anahtar sözcük olmaz; büyük/küçük harf duyarlı.
- Ders standardı: `ogrenci_sayisi` (snake_case, ASCII Türkçe); sabitler `MAX_OGRENCI`.

★ basamaklar: formülden koda (Örnek 3 ve 4)

Örnek 3 · tamsayı formül · H03_03_dikdortgen.c · 18–19

```c
18      int alan = en * boy;          /* A = en × boy       */
19      int cevre = 2 * (en + boy);   /* Ç = 2 × (en + boy) */
```

GERÇEK ÇIKTI · ÖRNEK 3

```
Dikdörtgenin eni (m) : 7
Dikdörtgenin boyu (m): 4
Alan   : 28 m²
Çevre : 22 m
```

Örnek 4 · double, const, %lf · H03_04_daire_alani.c · 11–15

```c
11      const double CM2_M2 = 0.0001;  /* const: tipli, değiştirilemez sabit */
12      double yaricap;
13
14      printf("Yarıçap (cm): ");
15      scanf("%lf", &yaricap);        /* double okumak için %lf */
```

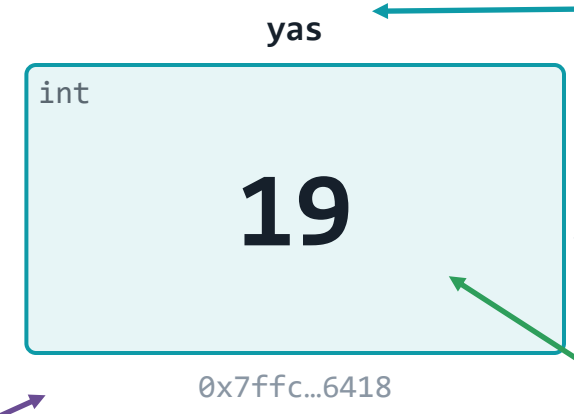
GERÇEK ÇIKTI · ÖRNEK 4

```
Yarıçap (cm): 5
Alan = 78.539816 cm² (varsayılan 6 basamak)
Alan = 78.54 cm²
Alan = 0.0079 m²
```

- `%f`: 6 basamak (varsayılan); `%.2f`: 2 basamak.
- `#define PI` ile `const` farkı: 3. bölümde.

Değişkenin dört niteliği: bellek kutusu

```
H03_K01_bildirim_atama.c · 18
18   int yas = 19;
```



1 AD

programcının verdiği isim: yas

2 TİP

int → 4 bayt, tamsayı

3 DEĞER

kutunun şu anki içeriği: 19

4 ADRES

&yas → bellekteki yeri (onaltılı)

Gösterim sözleşmesi (dönem boyunca): **ad üstte · değer içte · adres altta (gri) · tip sol üstte**

Bildirim, ilk değer atama ve atama

```
H03_K01_bildirim_atama.c · 16
16  int ogrenci_sayisi;
```

```
H03_K01_bildirim_atama.c · 17
17  ogrenci_sayisi = 85;
```

```
H03_K01_bildirim_atama.c · 18
18  int yas = 19;
```

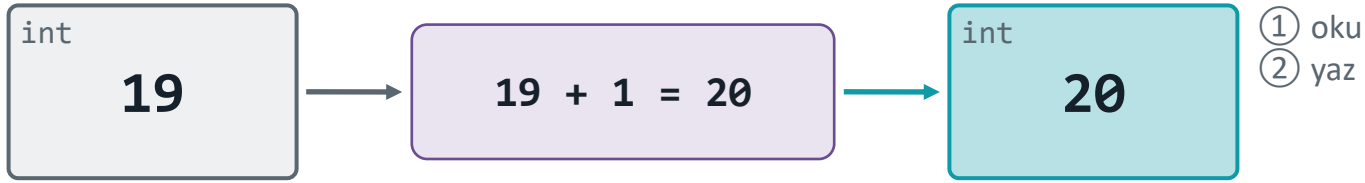


Bildirim: yer ayrılır, değer belirsiz (çöp)

Atama: kutuya yazılır, eski değer silinir

Bildirim + ilk değer: kutu doğar doğmaz dolar

```
H03_K01_bildirim_atama.c · 23
23  yas = yas + 1;
```



KAPUTUN ALTINDA

İlk değer atanmamış değişken: “çöp” değer

- `int toplam;` yer ayırır ama **temizlemez**: eski bitler durur.
- Standart: değer **belirsiz**; okumak **tanımsız davranışa** yol açabilir.
- Gerçek çalıştırma: 10 yerine **32775** yazdı (başka derlemede farklı olabilir).
- **Kural: bildirirken ilk değer verin.**

H03_HA3_ilk_deger_yok.c · 8–12

```
 8      int toplam;                  /* HATA: ilk değer verilmedi */
 9      int sayi = 10;
10
11      toplam = toplam + sayi;
12      printf("Toplam: %d\n", toplam);
```

GERÇEK ÇIKTI

```
$ ./ha3
Toplam: 32775
```

```
GCC -Wall: uyarı: 'toplam' is used uninitialized [-Wuninitialized]
```

TAHMİN ET

Bu program ne yazdırır? (Örnek 1)

H03_01_degisken_yazdir.c · 10–18

```
10      int yas = 19;               /* bildirim + ilk değer atama */
11      int ogrenci_sayisi;         /* yalnızca bildirim */
12
13      ogrenci_sayisi = 85;        /* atama: kutuya değer yazılır */
14      printf("Yaş: %d\n", yas);
15      printf("Sınıftaki öğrenci sayısı: %d\n", ogrenci_sayisi);
16
17      yas = yas + 1;              /* sağ taraf okunur, sonuç sola yazılır */
18      printf("Bir yıl sonra yaş: %d\n", yas);
```

A Yaş: 19 … Bir yıl sonra yaş: 19

B Yaş: 19 … Bir yıl sonra yaş: 20

C Derleme hatası: yas iki kez atanıyor

D Yaş: 20 … Bir yıl sonra yaş: 20

02

Veri tipleri ve tamsayıların bellekte temsili

sizeof, limits.h, LP64/LLP64, ikiye tümleyen, taşma ve küçük-uçluluk

Temel veri tipleri ve biçim belirteçleri

Tip	Ne saklar?	Değişmez	printf	scanf
`char`	Tek karakter / küçük tamsayı	`'A'`	`%c (%d)`	`%c`
`short`	Küçük tamsayı	`1200`	`%hd`	`%hd`
`int`	Tamsayı (varsayılan)	`42`	`%d`	`%d`
`unsigned int`	Negatif olmayan tamsayı	`42u`	`%u`	`%u`
`long`	Büyük tamsayı (platforma bağlı!)	`42L`	`%ld`	`%ld`
`long long`	Çok büyük (≥ 64 bit)	`42LL`	`%lld`	`%lld`
`float`	Tek duyarlıklı kesirli	`3.14f`	`%f`	`%f`
`double`	**Çift duyarlıklı kesirli (varsayılan)**	**`3.14`**	**`%f (%lf)`**	**`%lf (!)`**

printf: %f (ve %lf)

printf'e float, double'a yükseltilerek gider; C99'dan beri printf'te %lf de %f ile aynıdır.

scanf: %f ≠ %lf

scanf adres alır; float * için %f, double * için %lf. Karıştırmak tanımsız davranış.

Değişmez son ekleri

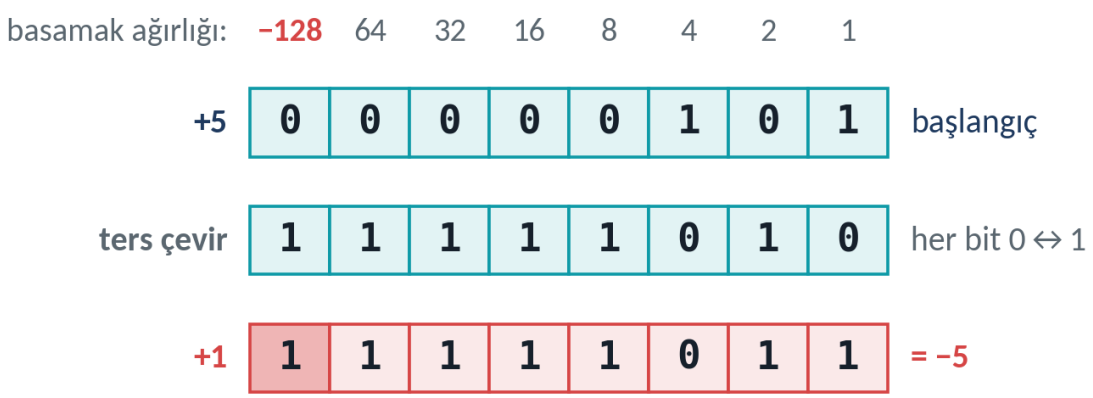
u işaretsiz, L long, LL long long, f float; eksiz 3.14 bir double'dır.

Bu makinede tipler kaç bayt? (Örnek 6'nın gerçek çıktısı)

Tip	Bayt	En küçük	En büyük	Windows x64
char	1	-128	127	1
short	2	-32768	32767	2
int	4	-2147483648	2147483647	4
unsigned int	4	0	4294967295	4
long	**8**	**-9223372036854775808**	**9223372036854775807**	**4 (!)**
long long	8	-9223372036854775808	9223372036854775807	8
float	4	−3.402823e+38	3.402823e+38	4
double	8	−1.797693e+308	1.797693e+308	8

Veri modelleri: Linux/macOS 64 bit **LP64** (long ve işaretçi 64 bit) · Windows 64 bit (MSVC, MinGW-w64) **LLP64** (long 32 bit, long long ve işaretçi 64 bit). Son sütun ölçüm değil, LLP64 modelinin tanımıdır. Standart yalnızca en küçük aralıkları belirler: int ≥ 16, long ≥ 32, long long ≥ 64 bit.

İkiye tümleyen: negatif sayılar nasıl saklanır?



Sağlama: 1111 1011 = −128 + 64 + 32 + 16 + 8 + 2 + 1 = −5

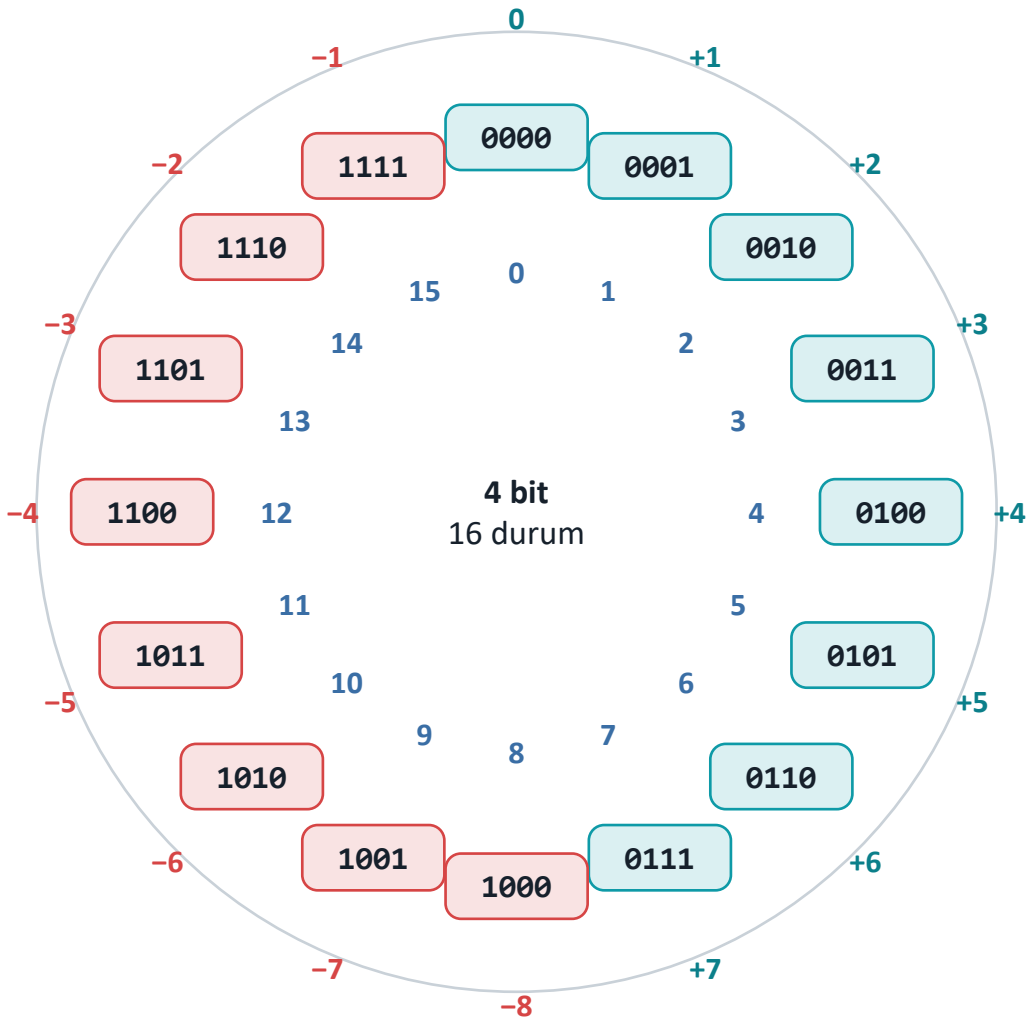
En soldaki bit −128 ağırlığındadır; 1 ise sayı negatiftir (işaret biti).

Neden bu gösterim? +5 ile −5 toplanınca: 0000 0101 + 1111 1011 = 1 0000 0000

9. bit (elde) sığmaz ve düşer → 0000 0000. Toplayıcı devre, çıkarma için ayrı bir devreye gerek duymaz.

- En soldaki bit **−128** ağırlıklı (8 bitte).
- −x = bitleri ters çevir, **1 ekle**.
- 5 + (−5): 9. bit düşer → **0**. Toplayıcı devre yeter.
- 1111 1111 = −1 (işaretli) = 255 (işaretsiz).

Taşma çemberi: sınırlı bit = kapalı bir çember



İşaretsiz: 1111 + 1 → 0000 (15 + 1 → 0)
C'de **tanımlı**: mod 16 aritmetiği (sarma; 32 bitte mod 2^{32})

İşaretli: 0111 + 1 → 1000 (7 + 1 → −8 görünür)
C'de **TANIMSIZ DAVRANIŞ**: sonuç güvence altında değil; derleyici taşma olmadığını varsayabilir

32 bitte aynı çember, 2^{32} durum:
`UINT_MAX + 1` → **0**
`INT_MAX + 1` → (bu makinede) **−2147483648**

Kilometre sayacı gibi: 999 999 km + 1 → 000 000

ÖRNEK MERDİVENİ ★★★

Deney: UINT_MAX + 1 ve INT_MAX + 1 (Örnek 11)

H03_11_tasma_deneyi.c · 11–23

```c
    unsigned int sayac = UINT_MAX;          /* 2^32 - 1 */
    printf("unsigned int en büyük : %u\n", sayac);
    sayac = sayac + 1;                      /* TANIMLI: mod 2^32 → 0 */
    printf("bir fazlası           : %u\n", sayac);

    int adim = 0;
    printf("INT_MAX'a kaç ekleyelim? ");
    scanf("%d", &adim);                     /* değer çalışma zamanında */

    int x = INT_MAX;                        /* 2^31 - 1 */
    printf("int en büyük          : %d\n", x);
    x = x + adim;                           /* taşarsa TANIMSIZ DAVRANIŞ */
    printf("INT_MAX + %d           : %d\n", adim, x);
```

UBSAN (TANIMSIZ DAVRANIŞ DENETLEYİCİSİ) İLE GERÇEK ÇIKTI

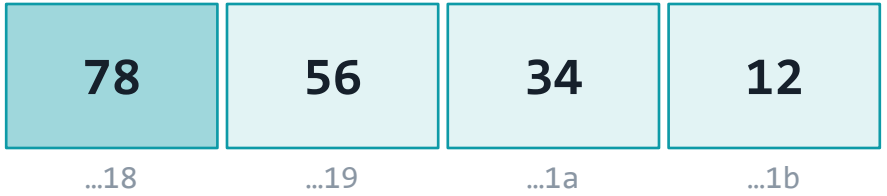
```
$ ./tasma_ub
unsigned int en büyük : 4294967295
bir fazlası           : 0
INT_MAX'a kaç ekleyelim? 1
int en büyük          : 2147483647
H03_11_tasma_deneyi.c:22:7: runtime error: signed integer overflow: 2147483647 + 1 cannot be
represented in type 'int'
INT_MAX + 1           : -2147483648
```

Kaputun altında: bayt sırası (küçük-uçluluk)

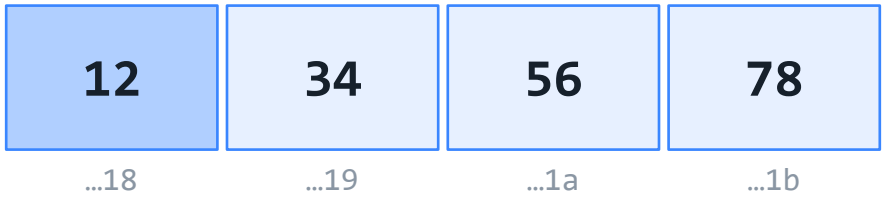
```
H03_K02_bayt_sirasi.c · 14
14  int sayi = 0x12345678;
```

4 bayt: **12 | 34 | 56 | 78** (12 en anlamlı bayt)

Küçük-uçlu (little-endian) · x86-64



Büyük-uçlu (big-endian) · ağ bayt sırası



Gerçek bellek (gdb, Örnek 9): `x = 42`, `d = 3.5`

GDB BELLEK DÖKÜMÜ

```
> x/4xb &x
0x7fffffff8f38:  0x2a 0x00 0x00 0x00
> x/8xb &d
0x7fffffff8f40:  0x00 0x00 0x00 0x00 0x00 0x00 0x0c 0x40
```

42 = 0x0000002A → bellekte **2a 00 00 00**; en anlamsız bayt önce.

*Bu hafta diyagramla gördük; baytlara C kodu içinden erişmeyi **H12'de işaretçilerle** göreceğiz.*

Aynı bitler, farklı yorum (Ek örnek 14)

H03_14_ayni_bitler.c · 10–18

```c
10      int i = -1;
11      unsigned int u = i;           /* aynı 32 bit, işaretsiz yorum */
12
13      printf("int olarak       : %d\n", i);
14      printf("unsigned olarak  : %u\n", u);
15      printf("onaltılı (%%x)    : %x\n", u);
16
17      signed char sc = -56;
18      unsigned char uc = sc;        /* aynı 8 bit: 1100 1000 */
```

GERÇEK ÇIKTI

```
int olarak       : -1
unsigned olarak  : 4294967295
onaltılı (%x)    : ffffffff
signed char      : -56
unsigned char    : 200
```

- `ffffffff`: **int** olarak −1, **unsigned** olarak 4294967295.
- `1100 1000`: signed char −56, unsigned char 200.
- Bellekte yalnızca bitler var; anlamını **tip** belirler.

03

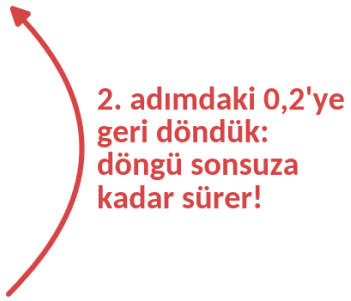
Kayan noktalı sayılar ve karakterler

IEEE 754, 0.1 + 0.2, ASCII ve sabitler

0,1 ikili tabanda neden sonlu yazılamaz?

0,1'i ikili tabana çevirme: kesirli kısmı art arda 2 ile çarp, tam kısmı yaz

1.	0,1 × 2	=	0,2	**0**
2.	0,2 × 2	=	0,4	**0**
3.	0,4 × 2	=	0,8	**0**
4.	0,8 × 2	=	1,6	**1**
5.	0,6 × 2	=	1,2	**1**
6.	0,2 × 2	=	0,4	**0**
7.	0,4 × 2	=	0,8	**0**
8.	0,8 × 2	=	1,6	**1**

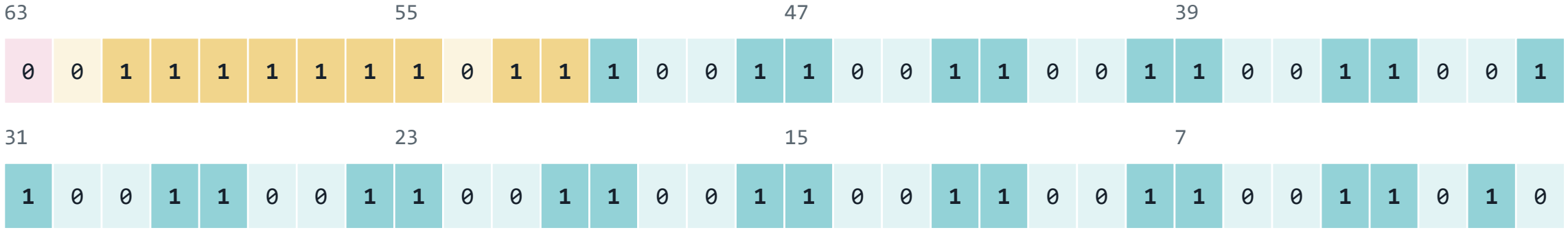


$0{,}1_{10} = 0{,}0\ 0011\ 0011\ 0011\ 0011\ \ldots_2$ (0011 sonsuza dek tekrar eder)

- Kesirli kısmı 2 ile çarp, tam kısmı yaz.
- 0,2 → 0,4 → 0,8 → 0,6 → **0,2** … döngü!
- $0{,}1_{10} = 0{,}0\mathbf{0011}0011\ldots_2$
- 1/3 onlukta nasıl sonsuzsa, **1/10 ikilide** sonsuzdur.
- Paydası 2'nin kuvveti olanlar (0.5, 0.25) **tam** saklanır.

0.1 bellekte: double'ın 64 biti

onaltılı: **0x3FB999999999999A** (`printf("%a", 0.1)` → `0x1.999999999999ap-4`)



İşaret s = 0	pozitif
Üs e = 01111111011_2 = 1019	1019 − 1023 = −4 → × 2^{-4}
Mantis 1,1001 1001 … 1010_2	≈ 1,6 · sonsuz 1001 örüntüsü 52 bitte kesilir, son bit yukarı yuvarlanır

Saklanan değer = 1,600000000000000888 × 2^{-4} = **0,1000000000000000055511…** (tam 0,1 değil!)

ÖRNEK MERDİVENİ ★★★

Deney: 0.1 + 0.2 ve %.20f (Örnek 12)

H03_12_kayan_nokta.c · 10–20

```c
10      double a = 0.1;
11      double b = 0.2;
12      double toplam = a + b;
13
14      printf("toplam (%%f)       : %f\n", toplam);
15      printf("toplam (%%.20f)    : %.20f\n", toplam);
16      printf("0.3 (%%.20f)       : %.20f\n", 0.3);
17      printf("0.1 (%%.20f)       : %.20f\n", a);
18      printf("toplam - 0.3      : %e\n", toplam - 0.3);
19      printf("0.3'e eşit mi?    : %d (1 evet, 0 hayır)\n", toplam == 0.3);
20      printf("0.1 (%%a)          : %a\n", a);
```

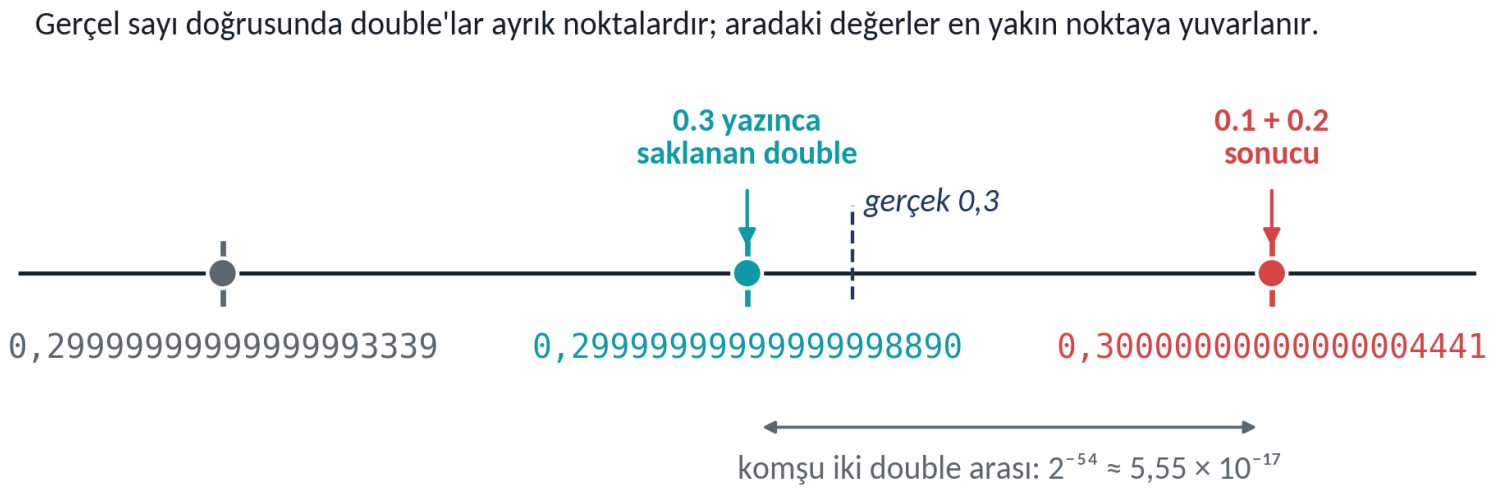
- `%f` 6 basamak gösterir, farkı **gizler**; `%.20f` gerçek açılımı gösterir.
- Fark tam bir adım (5.55e-17): `toplam == 0.3` sonucu **0**.

GERÇEK ÇIKTI (FLOAT SATIRI: DERS NOTU ÖRNEK 12)

```
toplam (%f)       : 0.300000
toplam (%.20f)    : 0.30000000000000004441
0.3 (%.20f)       : 0.29999999999999998890
0.1 (%.20f)       : 0.10000000000000000555
toplam - 0.3      : 5.551115e-17
0.3'e eşit mi?    : 0 (1 evet, 0 hayır)
0.1 (%a)          : 0x1.999999999999ap-4
```

Yazdırma **yuvarlar**;
bellekteki değer değişmez:
`%.2f` → 0.30
saklanan → 0.30000000000000004441

Kayan noktalı sayıları == ile karşılaştırmayın



- double'lar sayı doğrusunda **ayrık noktalardır**.
- 0.1 + 0.2, 0.3'ün **komşu** noktasına düşer (2^{-54}).
- Doğrusu: $|a - b| < \varepsilon$ (ör. 1e-9) — `fabs` H4'te.
- Para: kuruşları **tamsayı** tutun.
- Varsayılan kesirli tip: **double**.

char: karakter de bir sayıdır (Örnek 7)

Denetim ve noktalama

\0	\t	\n	\r	␣	!	%	+	-	.
0	9	10	13	32	33	37	43	45	46

Rakamlar

0	1	2	3	4	5	6	7	8	9
48	49	50	51	52	53	54	55	56	57

Büyük harfler

A	B	C	D	E	F	G	H	I	J	…	Z
65	66	67	68	69	70	71	72	73	74		90

Küçük harfler

a	b	c	d	e	f	g	h	i	j	…	z
97	98	99	100	101	102	103	104	105	106		122

char: 1 baytlık tamsayı · ekranda karakter, bellekte ASCII kodu · ␣ = boşluk

Eşlemeler

```
'A' = 65

'A' + 1 = 'B'

'a' − 'A' = 32

'7' − '0' = 7

'0' = 48 ≠ 0

'\0' = 0

'\n' = 10
```

GERÇEK ÇIKTI · H03_07

```
'A' karakterinin kodu: 65
'A' + 1 = 'B' (kod 66)
'a' = 97, 'A' = 65, fark = 32
'7' karakterinin kodu 55, 7 sayısı ise 7
'7' - '0' = 7 (rakam karakterinden sayıya)
'g' büyük harfe: 'G'
sizeof(char) = 1, sizeof('A') = 4
```

- `'A'` tek tırnak: karakter (65); `"A"` string.
- `'7' - '0'` = 7: rakamdan sayıya.
- `sizeof('A')` = 4: değişmez aslında int!

Sabitler: #define mi, const mı?

H03_K03_const_atama.c · 12–18

```
12  int main(void)
13  {
14      const double CM2_M2 = 0.0001;
15      CM2_M2 = 1.0;
16      printf("%f\n", CM2_M2);
17      return 0;
18  }
```

`#define PI 3.14159265358979`

- **Önişlemci** metni yerine koyar (derlemeden önce)
- Tipi ve adresi **yok**; sonuna `;` **konmaz**

`const double CM2_M2 = 0.0001;`

- **Derleyici** görür: tipli, salt okunur; adresi **var**
- Değiştirmeye çalışınca **hata** (altta) · ad: BUYUK_HARF

GERÇEK GCC İLETİSİ: PROGRAM DERLENMEZ

```
H03_K03_const_atama.c: In function 'main':
H03_K03_const_atama.c:15:12: error: assignment of read-only variable 'CM2_M2'
   15 |     CM2_M2 = 1.0;
      |            ^
```

04

printf ve scanf ile biçimli giriş/çıkış

Belirteçler, alan genişliği, & (adres), dönüş değeri ve giriş tamponu

printf biçim belirteçleri (gerçek çıktılar)

Belirteç	Tip	Örnek değer	Gerçek çıktı
`%d  %5d  %-5d  %05d`	int	`42`	`[42] [   42] [42   ] [00042]`
`%u`	unsigned int	`4294967295u`	`[4294967295]`
`%ld  %lld`	long, long long	`…807L, -5LL`	`[9223372036854775807] [-5]`
`%zu`	size_t	`sizeof(int)`	`[4]`
`%f  %.2f  %8.2f`	double	`3.14159`	`[3.141590] [3.14] [    3.14]`
`%e  %g`	double	`1234.5678`	`[1.234568e+03] [1234.57]`
`%c  %x  %o  %#x`	char / unsigned	`'A', 255`	`[A] [ff] [377] [0xff]`
`%p  %%`	void *, —	`(void *)&x`	`adres (değişken) · [%]`

Yuvarlama (gerçek glibc çıktısı)
`%.0f`: 2.5 → [2], 3.5 → [4] (tam ortada çift rakama)
`%.2f`: 0.125 → [0.12], 2.675 → [2.67] (2.675 bellekte 2.67499…)

Kural: belirteç = argümanın tipi
Uyuşmazlık **tanımsız davranıştır** (Hata Avcısı 2); `-Wall` uyarır.
`%p` ile yazılan adres her çalıştırmada değişir (ASLR).

Bir biçim belirtecinin parçaları

Genel biçim: %[bayrak][genişlik][.hassasiyet]dönüşüm



başlangıç	bayrak: sola yasla	alan genişliği	hassasiyet: virgülden sonra 2	dönüşüm: double

printf(biçim, değer)	gerçek çıktı (her kutu bir karakter)
`printf("[%8.2f]", 3.14159);`	[· · · · 3 . 1 4]
`printf("[%-8.2f]", 3.14159);`	[3 . 1 4 · · · ·]
`printf("[%08.2f]", 3.14159);`	[0 0 0 0 3 . 1 4]
`printf("[%5d]", 42);`	[· · · 4 2]
`printf("[%-5d]", 42);`	[4 2 · · ·]
`printf("[%+d]", 42);`	[+ 4 2]

· = boşluk

Her kutu bir karakter; · = boşluk. Tüm çıktılar gerçek.

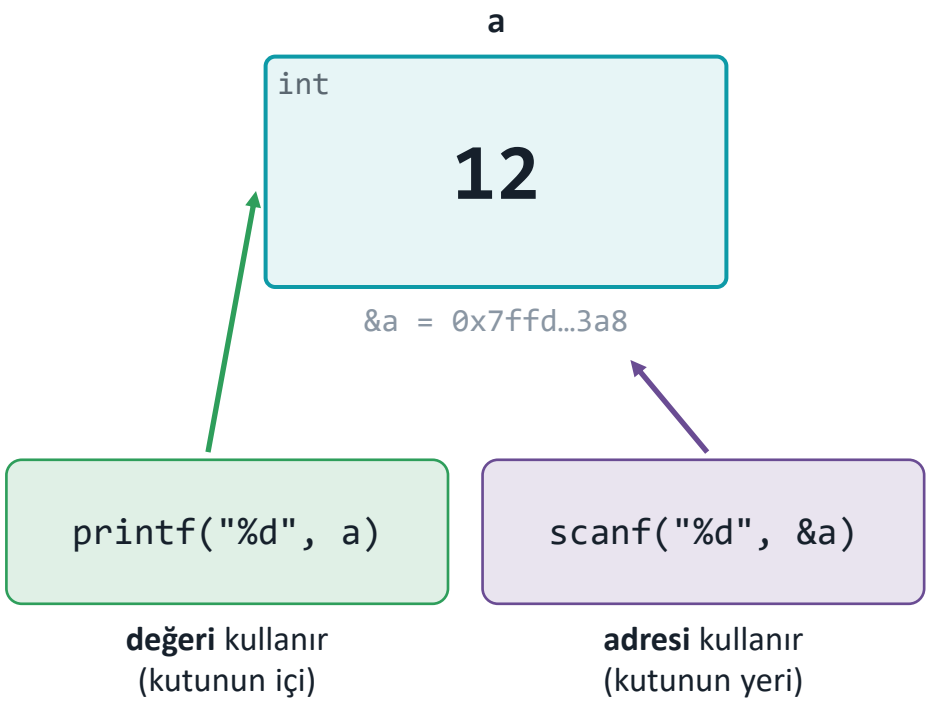
scanf neden & ister? (Örnek 2)

H03_02_iki_sayi_toplam.c · 10–18

```
10      int a;
11      int b;
12
13      printf("Birinci sayı: ");
14      scanf("%d", &a);            /* &a: "a'nın adresine yaz" */
15      printf("İkinci sayı : ");
16      scanf("%d", &b);
17
18      int toplam = a + b;
```

GERÇEK ÇIKTI

```
Birinci sayı: 12
İkinci sayı : 30
12 + 30 = 42
```



scanf değeri **yazacağı yeri** bilmeli → &a

scanf ne döndürür? (Ek örnek 13)

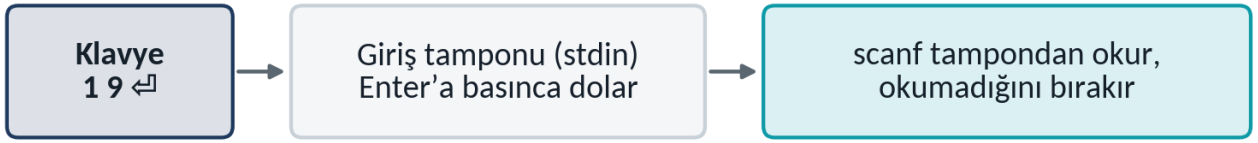
H03_13_scanf_donus.c · 14–17

```
14      int okunan = scanf("%d %d", &a, &b);   /* başarıyla okunan öge sayısı */
15
16      printf("scanf'in dönüş değeri: %d\n", okunan);
17      printf("a = %d, b = %d\n", a, b);
```

Girdi	Dönüş	a	b
12 30	2	12	30
12 abc	1	12	0
abc	0	0	0
Ctrl+D (EOF)	-1	0	0

- Dönüş değeri = **başarıyla okunan öge sayısı**.
- Hiçbir şey okunmadan dosya sonu: **EOF** (−1).
- Okuma başarısızsa değişkene **dokunulmaz** → ilk değer verin (a ve b 0 ile başlatıldı).
- H6'dan itibaren her `scanf`'in dönüş değerini **denetleyeceğiz**.

Giriş tamponu ve " %c" çözümü (Ek örnek 15)



Adım 1

```
scanf("%d", &yas);
```

1	9	\n

"19" → yas = 19;
'\n' tamponda KALIR

Adım 2 (sorunlu)

```
scanf("%c", &harf1);
```

\n

bekleyen '\n' okunur:
harf1 = 10, istem atlanır!

Adım 2 (çözüm)

```
scanf(" %c", &harf1);
```

\n	B	\n

boşluk, beyaz karakterleri
atlar → harf1 = 'B'

okundu · istenmeden okundu · atlandı · tamponda bekliyor

GERÇEK OTURUM · H03_15

```
Yaşınız: 19
Bir harf (1. deneme): Bir harf (2. deneme): B

yas = 19
harf1 kodu = 10 (10, '\n' karakterinin kodudur)
harf2 = 'B'
```

- %d sayıyı okur, `'\n'` tamponda **kalır**.
- %c bekleyen `'\n'`i hemen alır: istem **atlanır**.
- `" %c"`: baştaki boşluk beyaz karakterleri **atlar**.

05

Örnek merdiveni: amiral örnekler

Celsius → Fahrenheit, takas, adresler ve VKİ

Haftanın örnek merdiveni

#	Örnek	Yeni öge	Düzey
1	Değişken bildirip değerini yazdırma	Bildirim, ilk değer, printf %d	★
2	Klavyeden iki sayı okuyup toplamını yazdırma	scanf ve &	★
3	Dikdörtgenin alanı ve çevresi	Formülün koda aktarılması	★
4	Dairenin alanı	double, const/#define PI, %.2f	★
5	Celsius → Fahrenheit dönüşümü	9/5 tamsayı bölmesi tuzağına ilk temas	★★
6	Makinenizde veri tipi boyutları tablosu	sizeof, limits.h; platform farkı	★★
7	Karakter ↔ ASCII kodu dönüşümü ('A' + 1)	char'ın sayısal doğası	★★
8	İki değişkenin değerlerini takas etme	Geçici değişken (üçüncü bardak analojisi)	★★
9	Değişkenlerin adreslerini yazdırma	&, %p; bellek hücresi modeli	★★
10	Vücut kitle indeksi hesaplayıcı	Bütünleştirici: giriş, işlem, biçimli çıktı	★★
11	Tamsayı taşması deneyi (INT_MAX + 1)	İkiye tümleyen; taşma	★★★
12	0.1 + 0.2 deneyi ve %.20f	Kayan nokta hassasiyeti	★★★

Amiral örnek: Celsius → Fahrenheit (Örnek 5)

H03_05_celsius_fahrenheit.c · 8–21

```c
int main(void)
{
    double celsius = 0.0;          /* okuma başarısız olursa 0 kalır */

    printf("Sıcaklık (°C): ");
    scanf("%lf", &celsius);

    double f_hatali = celsius * (9 / 5) + 32;     /* 9 / 5 → 1 (!) */
    double f_dogru = celsius * 9.0 / 5.0 + 32;    /* 9.0 / 5.0 → 1.8 */

    printf("Hatalı formül: %.1f °F\n", f_hatali);
    printf("Doğru formül : %.1f °F\n", f_dogru);
    return 0;
}
```

GİRDİ	celsius (double, °C)
İŞLEM	f = c × 9 / 5 + 32
ÇIKTI	f, bir ondalıkla (%.1f)

- **15. satır:** `9 / 5` = **1** (tamsayı bölmesi!) → F = C + 32
- **16. satır:** `9.0 / 5.0` = 1.8
- Derleyici **uyarmaz**: geçerli C; hatayı ancak **test** yakalar.

Algoritma: OKU celsius → f ← celsius × 9 / 5 + 32 → YAZ f
Tahmin: 36.6 °C için iki formül ne yazdırır? Çıktı ve testler sonraki slaytta.

Örnek 5'in test tablosu (gerçek çalıştırmalar)

Dikkat: **0 °C** girdisinde iki formül de 32.0 verir. Yalnızca 0 ile test etseydik hatayı **göremezdik**.

Girdi	Beklenen °F	Hatalı formül	Doğru formül	Sonuç
36.6 (normal)	`97.9`	`68.6`	`97.9`	✓ doğru formül
100 (kaynama noktası)	`212.0`	`132.0`	`212.0`	✓ doğru formül
-40 (iki ölçeğin kesiştiği nokta)	`-40.0`	`-8.0`	`-40.0`	✓ doğru formül
0 (donma noktası)	**`32.0`**	**`32.0`**	**`32.0`**	**✓ doğru formül**
otuz (hatalı girdi)	`hata mesajı`	`32.0`	`32.0`	uyarı yok → H5–H6

GERÇEK ÇALIŞTIRMA · 36.6

```
$ ./celsius
Sıcaklık (°C): 36.6
Hatalı formül: 68.6 °F
Doğru formül : 97.9 °F
```

GERÇEK ÇALIŞTIRMA · HATALI GİRDİ

```
$ ./celsius
Sıcaklık (°C): otuz
Hatalı formül: 32.0 °F
Doğru formül : 32.0 °F
```

Takas ve izleme tablosu (Örnek 8)

H03_08_takas.c · 10–18

```c
10      int a = 5;
11      int b = 9;
12      printf("Önce : a = %d, b = %d\n", a, b);
13
14      int gecici = a;    /* 1. adım: a'yı boş bardağa dök */
15      a = b;             /* 2. adım: b'yi a'ya dök        */
16      b = gecici;        /* 3. adım: boş bardaktakini b'ye dök */
17
18      printf("Sonra: a = %d, b = %d\n", a, b);
```

Adım	Satır	a	b	gecici
1	10–11	5	9	–
2	14	5	9	5
3	**15**	**9**	**9**	**5**
4	16	9	5	5

GERÇEK ÇIKTI

```
Önce : a = 5, b = 9
Sonra: a = 9, b = 5
```

- `a = b; b = a;` yazsaydık: a'nın 5'i **kaybolurdu** → ikisi de 9.
- Üçüncü bardak analojisi: iki bardağın içeriğini değiştirmek için boş bir bardak gerekir. Takas → AP-2'de **sıralama** algoritmalarının temel işlemi.

Değişkenlerin adresleri (Örnek 9)

H03_09_adresler.c · 10–18

```c
    int x = 42;
    int y = 7;
    double d = 3.5;
    char c = 'Z';

    printf("x: değer %4d   adres %p   boyut %zu\n", x, (void *)&x, sizeof(x));
    printf("y: değer %4d   adres %p   boyut %zu\n", y, (void *)&y, sizeof(y));
    printf("d: değer %4.1f   adres %p   boyut %zu\n", d, (void *)&d, sizeof(d));
    printf("c: değer %4c   adres %p   boyut %zu\n", c, (void *)&c, sizeof(c));
```

1. ÇALIŞTIRMA

```
x: değer   42   adres 0x7ffcf96cd208   boyut 4
y: değer    7   adres 0x7ffcf96cd20c   boyut 4
d: değer  3.5   adres 0x7ffcf96cd210   boyut 8
c: değer    Z   adres 0x7ffcf96cd207   boyut 1
```

2. ÇALIŞTIRMA

```
x: değer   42   adres 0x7ffdfc93ce28   boyut 4
y: değer    7   adres 0x7ffdfc93ce2c   boyut 4
d: değer  3.5   adres 0x7ffdfc93ce30   boyut 8
c: değer    Z   adres 0x7ffdfc93ce27   boyut 1
```

- Adresler **her çalıştırmada değişir** (ASLR: güvenlik için rastgele yerleşim); aralarındaki **düzen** aynı kalır: x → y 4 bayt, y → d 4 bayt.
- `%p` bir `void *` bekler: `(void *)&x` (H12'de açıklanacak).

Amiral örnek: vücut kitle indeksi (Örnek 10)

H03_10_vki.c · 18–28

```c
18      double boy_m = boy_cm / 100.0;          /* cm → m */
19      double vki = kilo / (boy_m * boy_m);    /* VKİ = kg / m² */
20
21      printf("\n========== VKİ RAPORU ==========\n");
22      printf("Kilo             : %7.1f kg\n", kilo);
23      printf("Boy              : %7.2f m\n", boy_m);
24      printf("Vücut kitle ind. : %7.2f kg/m²\n", vki);
25      printf("---------------------------------\n");
26      printf("Başvuru (DSÖ, yetişkin):\n");
27      printf("  18,5 altı zayıf  | 18,5–24,9 normal\n");
28      printf("  25–29,9 fazla kilolu | 30 ve üstü obez\n");
```

GERÇEK ÇIKTI (70, 175)

```
Kilonuz (kg): 70
Boyunuz (cm): 175

========== VKİ RAPORU ==========
Kilo             :    70.0 kg
Boy              :    1.75 m
Vücut kitle ind. :   22.86 kg/m²
---------------------------------
Başvuru (DSÖ, yetişkin):
  18,5 altı zayıf  | 18,5–24,9 normal
  25–29,9 fazla kilolu | 30 ve üstü obez
```

- Parantez şart: `kilo / (boy_m * boy_m)`.`%7.2f` ile sütunlar hizalı.
- Testler: boy 0 → **inf**; sayı yerine yazı → **-nan** (0/0). Çökmez ama anlamsız: girdi doğrulama H5'te.

Hata Avcısı 1: scanf’te & unutmak

H03_HA1_scanf_ampersand.c — HATALI

```
 8      int yas = 0;
 9
10      printf("Yaşınız: ");
11      scanf("%d", yas);            /* HATA: & unutuldu */
12      printf("Yaş: %d\n", yas);
```

H03_HA1_scanf_ampersand_duzeltilmis.c — DOĞRU

```
 8      int yas = 0;
 9
10      printf("Yaşınız: ");
11      scanf("%d", &yas);           /* DOĞRU: yas'ın adresi verilir */
12      printf("Yaş: %d\n", yas);
```

GCC -Wall uyarısı (gerçek)

```
format '%d' expects argument of type
'int *', but argument 2 has type 'int'
[-Wformat=]
```

Uyarı hata değildir: program derlenir, ama çalışınca **çöker**.

GERÇEK ÇALIŞTIRMA (HATALI)

```
$ ./ha1
Yaşınız: 25
bash: line 1: Segmentation fault      ./ha1
[çıkış kodu: 139]
```

GERÇEK ÇALIŞTIRMA (DOĞRU)

```
$ ./ha1d
Yaşınız: 25
Yaş: 25
[çıkış kodu: 0]
```

Hata Avcısı 2: belirteç–tip uyuşmazlığı

H03_HA2_bicim_uyumsuzlugu.c

```c
int main(void)
{
    double ortalama = 77.5;
    int not_sayisi = 2;

    printf("Ortalama  : %d\n", ortalama);     /* HATA: double için %d */
    printf("Not sayısı: %f\n", not_sayisi);   /* HATA: int için %f */
    return 0;
}
```

GERÇEK ÇIKTI

```
$ ./ha2
Ortalama  : 1840679640
Not sayısı: 77.500000
```

```
uyarı: format '%d' expects argument of type 'int', but argument 2 has type 'double' [-Wformat=]
uyarı: format '%f' expects argument of type 'double', but argument 2 has type 'int' [-Wformat=]
```

- Kaputun altı: x86-64'te int argümanlar **genel yazmaçlarda**, double argümanlar **XMM yazmaçlarında** geçer.
- %d yanlış yazmaçtan rastgele bir sayı okudu; %f önceki çağrıdan kalan **77.5**'i buldu. Sonuç rastlantısaldır: **tanımsız davranış**.

Hata Avcısı 3: derleyicinin (her zaman) yakalamadıkları

İlk değer yok

- `int toplam;` → 10 yerine **32775**
- `-Wall`: “is used uninitialized”
- Çözüm: `int toplam = 0;`

Tamsayı bölmesi

- `(5 + 2) / 2` → **3** (uyarı yok)
- `ortalama = 3.000000`
- Çözüm: `/ 2.0`

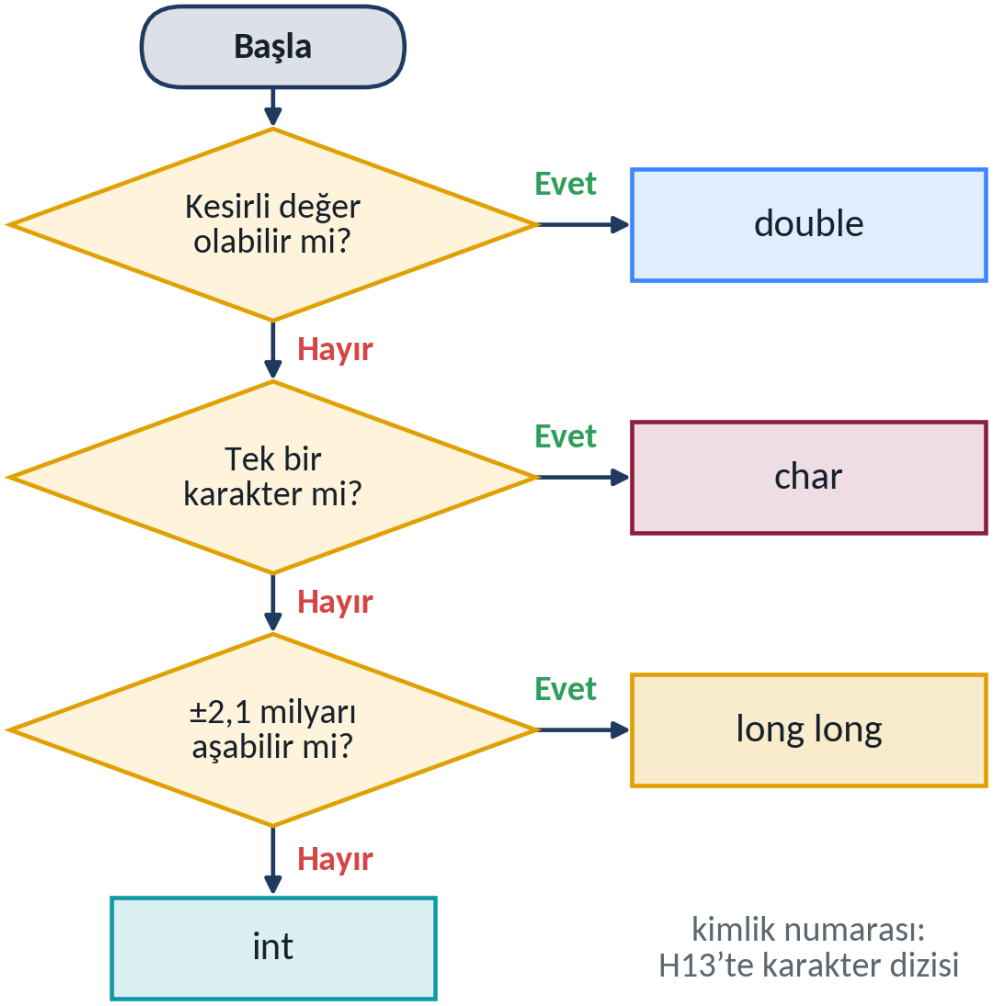
== ile kayan nokta

- `0.1 * 3 == 0.3` → **0** (uyarı yok)
- Fark: 5.55e-17
- Çözüm: toleransla karşılaştır

Tampondaki '\n'

- `scanf("%c")` istemi atlar (uyarı yok)
- harf1 = 10 ('\n')
- Çözüm: `scanf(" %c", &c)`

Mühendis gözüyle: tip seçimi bir sözleşmedir



kimlik numarası:
H13'te karakter dizisi

- **Aralığı önceden hesaplayın:** 30 gün ms cinsinden 2 592 000 000 > `INT_MAX`; 32 bitlik ms sayacı ≈ 24,8 günde dolar.
- **Kimlik ≠ sayı:** öğrenci no, T.C. kimlik no → aritmetik yok (H13'te karakter dizisi).
- **Boyut varsaymayın:** `long` 8 / 4 bayt; tam genişlik için `<stdint.h>` (`int32_t`).
- **Derleyiciyi kullanın:** `-Wall -Wextra -pedantic`, `-fsanitize=undefined`.
- **Sınır değerleri test edin:** 0, en büyük, negatif, hatalı girdi.

MERAK KUTUSU

Derleyici işaretli taşmanın olmayacağını varsayar

H03_17_tasma_varsayimi.c · 17–19

```
17      int sonuc = x + adim;                /* taşma: tanımsız davranış */
18      printf("x + adim         = %d\n", sonuc);
19      printf("x + adim > x mi? = %d (1 evet, 0 hayır)\n", x + adim > x);
```

NORMAL DERLEME · GİRDİ 1

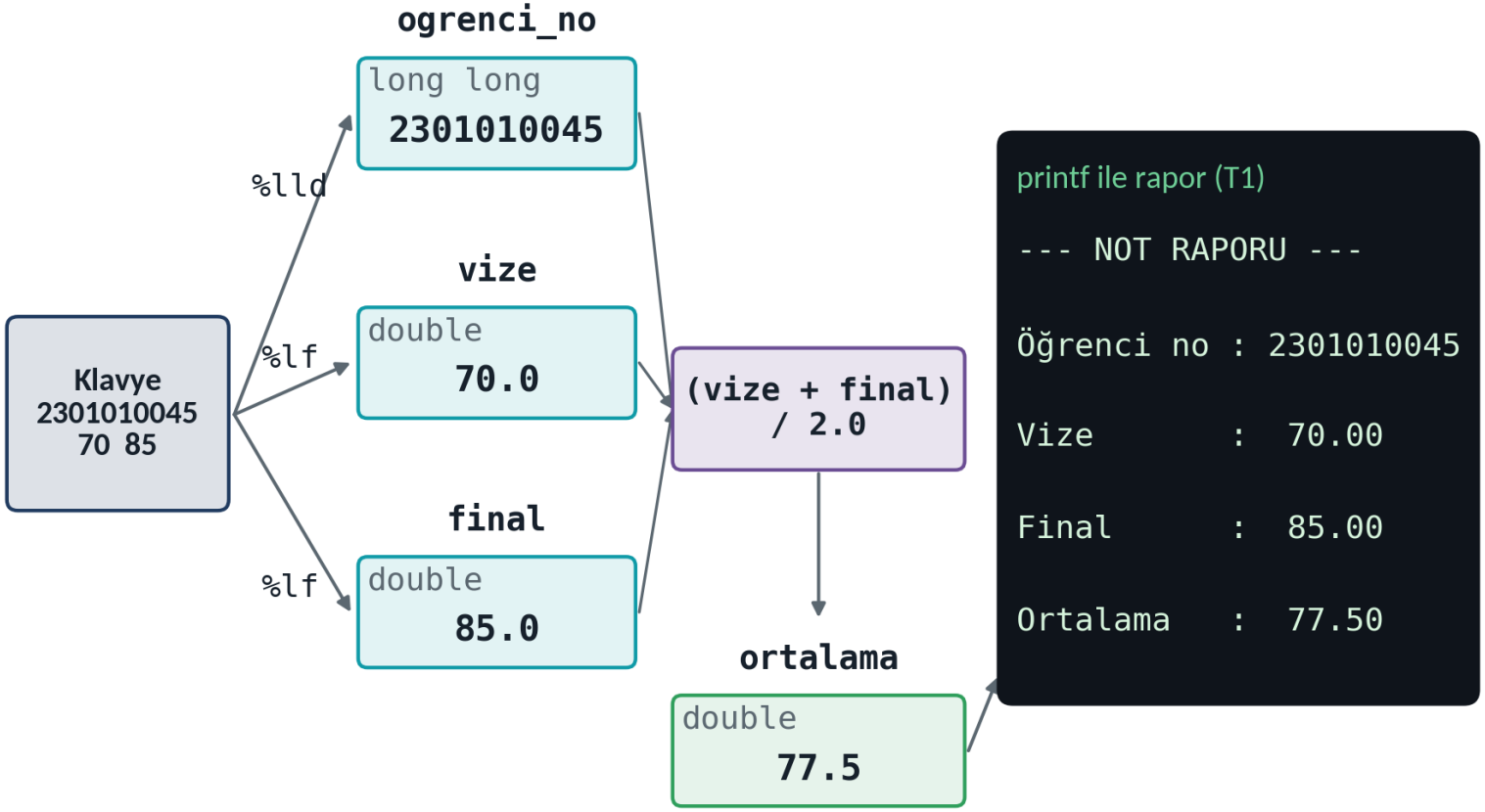
```
$ ./varsayim
INT_MAX'a kaç ekleyelim? 1
x + adim         = -2147483648
x + adim > x mi? = 1 (1 evet, 0 hayır)
```

- Toplam **–2147483648** ama `x + adim > x` **1**: GCC ifadeyi `adim > 0`'a sadeleştirdi.
- Taşma tanımsız → derleyici "olmaz" varsaymakta **özgür**.

UBSAN İLE: İKİ TAŞMA RAPORLANIR, KARŞILAŞTIRMA 0

```
$ ./varsayim_ub
INT_MAX'a kaç ekleyelim? 1
H03_17_tasma_varsayimi.c:17:9: runtime error: signed integer overflow: 2147483647 + 1 cannot be
represented in type 'int'
x + adim         = -2147483648
H03_17_tasma_varsayimi.c:19:60: runtime error: signed integer overflow: 2147483647 + 1 cannot be
represented in type 'int'
x + adim > x mi? = 0 (1 evet, 0 hayır)
```

Mini-OBS v0.3: ilk gerçek veri



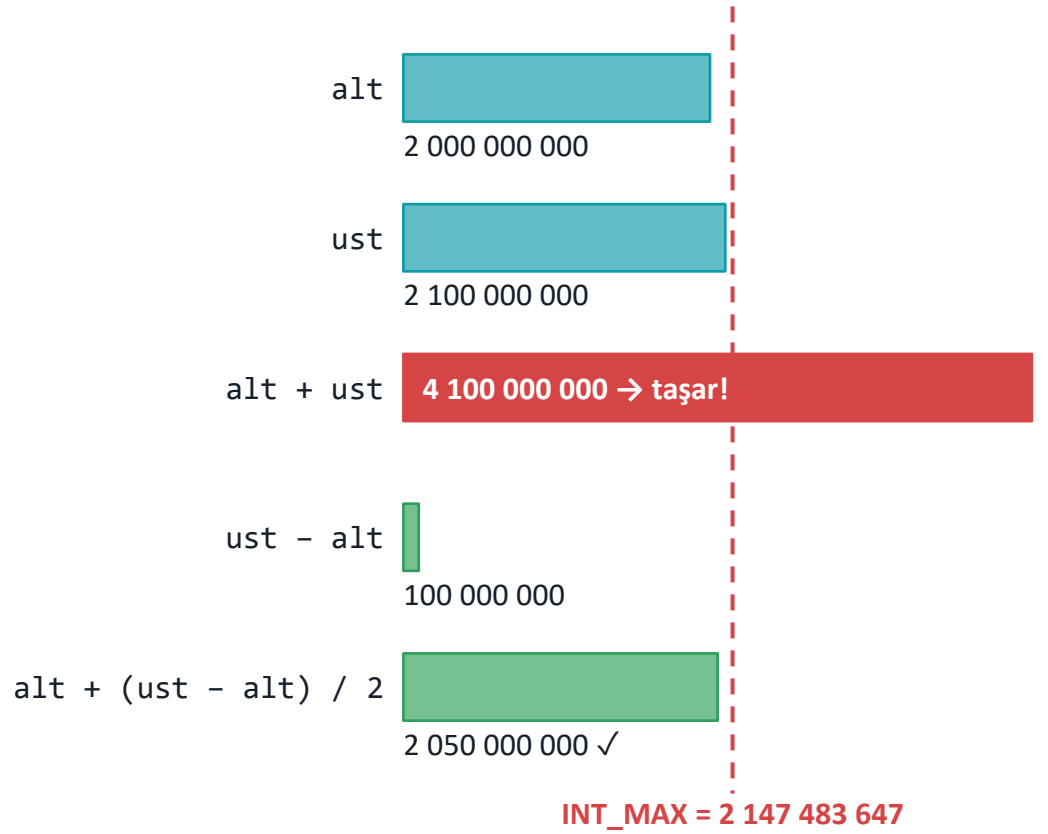
- Tek öğrenci: **no, vize, final** oku; **basit ortalama**; biçimli rapor.
- No için `long long` + `%lld` (10 hane > `INT_MAX`).
- Notlar `double`: `%lf` ile oku, `%6.2f` ile yaz; `/ 2.0`.
- Kabul: sıfır uyarı · hizalı rapor · tüm değişkenlere ilk değer.
- Test: 70, 85 → **77.50**; 72.5, 88 → **80.25**.

AP-2 KÖPRÜSÜ

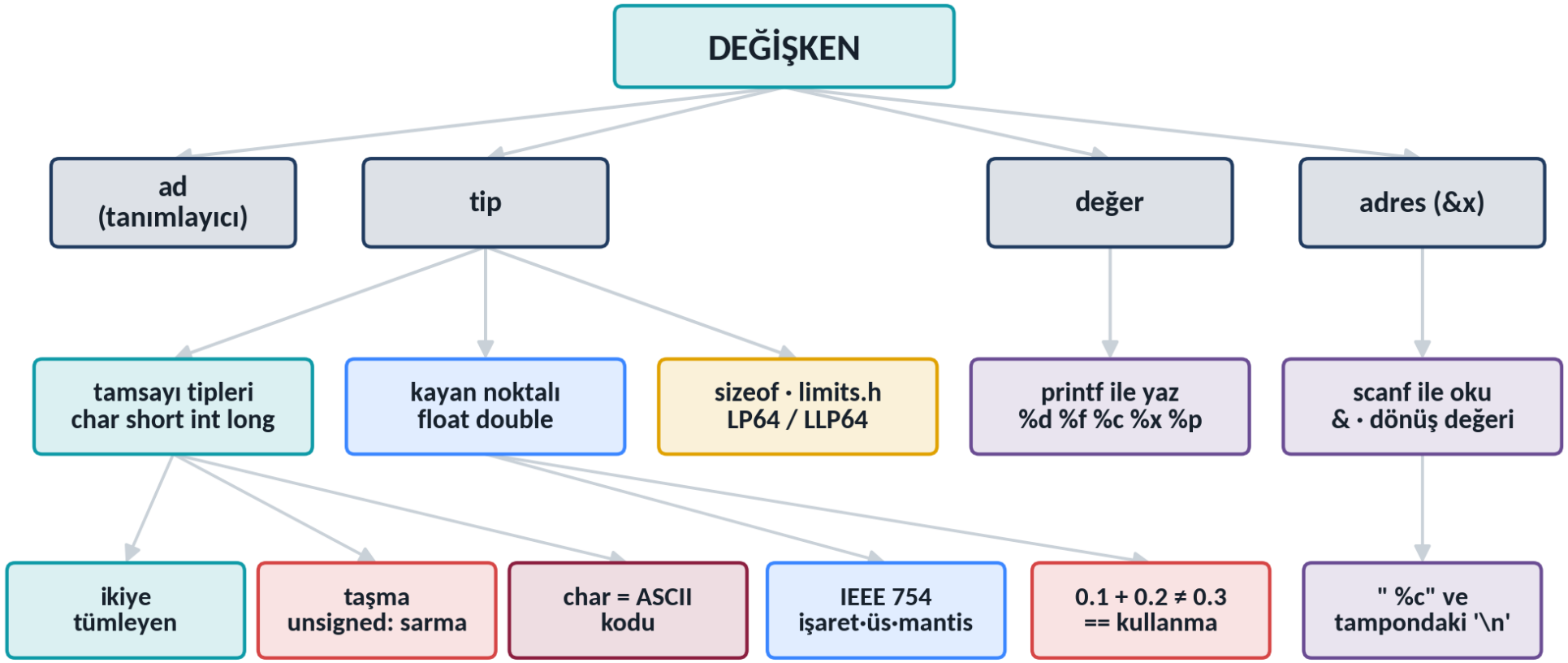
AP-2 köprüsü: taşma ve algoritma doğruluğu

- Kâğıt üzerinde doğru bir algoritma, bir **ara sonuç taştığı** için yanlış sonuç verebilir.
- Klasik örnek: `(alt + ust) / 2` taşabilir; `alt + (ust - alt) / 2` taşmaz. AP-2'de **ikili arama**nın orta nokta hesabında.
- Milyarlarca adımı sayan bir **sayaç** `int`'e sığmayabilir.
- Bu hafta ektiğimiz tohum: **aralık bilinci**.

alt = 2 000 000 000, ust = 2 100 000 000 (ikisi de int'e sığar)



Haftanın özeti: kavram haritası



Oklar "içerir / ilişkilidir" anlamındadır. Aynı bitler, farklı tipte farklı değer demektir.

Değişken → ad, tip, değer, adres; tip → tamsayı (ikiye tümleyen, taşma), kayan nokta (IEEE 754), char (ASCII); değer ve adres → printf / scanf

BÜYÜK SORU

Büyük Sorunun yanıtı

Neden 0.1 + 0.2 ≠ 0.3?

- 0,1; 0,2; 0,3 ikili tabanda **sonsuz** kesirdir.
- Her biri en yakın double'a **yuvarlanır**.
- Toplam, 0.3'ün **komşu** double'ına düşer (fark 2^{-54}).
- Hata değil: sonlu bitin kaçınılmaz sonucu → `==` kullanmayın.

Sayaç neden eksiye düşer?

- 32 bitlik işaretli sayaç en fazla **2 147 483 647**.
- İkiye tümleyende sonraki örüntü **−2 147 483 648**.
- C'de işaretli taşma **tanımsız davranıştır**: sarmaya güvenilmez.
- Çözüm: aralığı yeten tip (64 bit) — Gangnam, 2038.

Bilgisayar yanlış hesaplamıyor: **sonlu bitle, kurallarına uygun** çalışıyor.
Mühendisin işi: tipi ve temsili **bilerek** seçmek (Patriot · Gangnam Style · 2038).

ÇIKIŞ BİLETİ

Dersten çıkmadan önce üç soru

1 `int x = 10; double y = x / 4;` sonrası `printf("%.2f", y);` ne yazdırır? Neden?

2 `scanf("%d", n);` satırındaki hata nedir? Derleyici ne der, program ne yapar?

3 `INT_MAX + 1` ile `UINT_MAX + 1` arasındaki fark nedir? Hangisi C'de tanımlıdır?

Haftaya hazırlık ve kaynaklar

- **H4 · Operatörler, ifadeler ve tip dönüşümleri:** tamsayı bölmesi, %, öncelik, bit operatörleri, cast.
  - Büyük Soru: tek bir tip dönüşümü bir roketi nasıl düşürebilir? (Ariane 5, 1996)
- **Atölye teslimi:** kişisel bilgi kartı, birim dönüştürücü; keşif: Örnek 6'yı kendi makinenizde çalıştırın (LP64 mi, LLP64 mü?).
- **Mini-OBS v0.3:** kabul ölçütleri ve 6 test senaryosu ders notunda (Bölüm 7).
- **Web crash course:** bit görüntüleyici, taşma çemberi, printf oyun alanı ve tampon simülasyonu; 10 soruluk quiz.
- **Okuma:** Deitel & Deitel (2022), veri tipleri ve biçimli giriş/çıkış; Bryant & O'Hallaron (2016), Bölüm 2 (bilginin temsili).